SEASUG Paper 070-2026

Leveraging SAS to Automatically Assist with Code Reviews: An Academic Application

Rachel Winpisinger, North Carolina State University
Jonathan Duggins, North Carolina State University

Abstract

Code reviews are ubiquitous in any industry that employs – or trains! – programmers. In some cases, automated tools can help support reviewers by executing the program and scraping the log for controlled terminology such as error and warning messages. Programming courses in academia pose two additional challenges to an already time-consuming, complex topic: multiple submissions, often both within and across students, and frequent changes as to who is performing the code review due to turnover in instructional teams (instructors and/or grading assistants) from semester to semester. This inherent variability means creating a standardized code review process – especially one that works in a classroom or internship training setting – is quite challenging.

In this paper, we present a method that employs SAS and batch files using DOS commands to create a code review workflow. We use SAS programming tools ranging from simple data cleaning via `DATA` step functions to intermediate `DATA` step tools like arrays and integrate them with more advanced macro-based approaches. Results are tabulated and displayed in a color-coded spreadsheet via `PROC REPORT`. The automatic code-review tool is available on GitHub from the following repository jwduggins/AutograderPublic and includes demo submissions that allow stress testing of this code-review tool.

Motivation & Applications

Our motivation for the creation of an automated code review tool is two-fold: student to instructor ratio and consistency. As class sizes continue to rise and education funding continues to fall, we are forced to make concessions in how we assess students and provide feedback on those assessments. It is no longer reasonable to expect an instructor or teaching assistant (TA) to review all student submissions and provide substantive feedback. Increased class sizes also put pressure on university infrastructure, like the file servers that contain course materials (e.g., large data sets), causing programs to take longer to run. This, coupled with TAs that are spread across more sections of courses, means that too much of their time is spent on running code and reviewing logs and not on providing meaningful feedback. Providing this meaningful feedback consistently is also an issue as instructors and TAs rotate, meaning that course standards can vary and student learning is negatively impacted. W. Edwards Deming would be disappointed if we did not work to find a way to train statisticians and statistical programmers while implementing some quality control methods.

While our motivation has a clear application in academia, our goals, and the techniques we used to accomplish them, have applications in essentially any industry. We use batch files to identify all programs that need to be executed which is necessitated due to multiple students submitting files for the same assignment. We also use a batch program to execute each submission in independent calls to the SAS kernel – a necessity to ensure that each student is graded independently of each other student. In other applications, we can use batch files to identify all programs that need to be run on a single, large-scale project like a clinical trial and then independently execute them so that issues in one file do not affect the execution and results of another program. These batch files, along with other code discussed throughout this paper, make explicit use of the fact that we run SAS in a Windows environment. If you plan to implement this in a Linux or cloud environment, you will need to modify those programming elements.

Macro variables, and macro programs in general, allow us to apply consistent rules to each submission. This includes editing the files since we do not have access to student drives and they don't have access to ours; thus we ensure paths are adjusted for our local networks' peculiarities. These macros also allow us

to capture the logs and read them in as data, so that we can leverage `DATA` step logic and functions to search for any issues that appear in the log. This again is applicable in industry since SAS errors and warnings are standardized and commands like `PUT` and `PUTLOG` allow users to place custom messages – which can then be searched for – to identify any issues. We use a broad set of tools to carry this out: arrays, regular expressions, conditional logic, `RETAIN` statements, and `DATA` step functions.

In each section of this paper, we discuss one of the key elements of our code review program, i.e., our autograder. This approach is designed to help readers see how the pieces fit together and identify any pieces they may be able to apply in their own code review process. Note that due to the length of the overall program, in many cases we only display select portions of code. Readers may always refer to the full code at the accompanying GitHub site.

Key Global Options

In any application of a code review process, the user must specify the path where programs to be reviewed are stored. Given our use case – students submitting programs to be graded as part of their coursework – we refer to these files as *submissions*. Program 1 below uses a macro variable for storing this and other key variables needed throughout our program. As with most code review processes, the code being reviewed will also be generating output and so a path is required for this as well. In our case, we have assumed this path already exists rather than using SAS to create it. However, if the user desires, this is a fairly straightforward addition using the `DLCREATEDIR` option. For more information on this, please refer to the SAS documentation.

In our case, grading assistants may be faced with a scenario where they need to provide reviews on multiple different assignments for the same students. To accommodate this, we created a four-character prefix to uniquely identify all assignments within a course. This prefix is set at the beginning of the program to ensure that only files associated with the current project are processed.

The last macro variable we create, which stores the path to the SAS executable file on the current installation of SAS, is necessary for creating a file that executes each student submission in an independent environment – this is covered in the section on executing submissions.

Program 1: Setting Up the Environment

```
%let dir=%str(S:\SEASUG 2026\Autograder\Demo); ❶
%let out=%str(S:\Documents\Example\Results); ❶
%let Assignment = HW00; ❶
%let SasExeDir = %str(%sysget(sasroot)); ❶

options noxwait; ❷
ods trace off; ❸
```

❶ Create macro variables to store key concepts for later use. Each variable is described below during their first use.

❷ Allows the Windows Command Prompt window to close without user input. The default is `XWAIT` which would interfere with the automatic process described later in this paper.

❸ Turn off `ODS TRACE` to avoid adding extra information to the logs before they are scraped.

Collecting & Indexing All SAS Programs

In our workflow, student submissions are sent to a third-party software – our course management system (CMS) – and we can only export them as a ZIP file. We then extract the ZIP file to a path of our choice and Program 1 shows the creation of a macro variable, `Dir`, to store the location of these extracted

submissions. Unfortunately for us, when the CMS creates the ZIP file, it does so by placing each student's submission into a separate folder. Each folder is named using a generic prefix ("submission_") followed by a nine digit number. Thus, while the folder names are unique, they are not named using the student name, submission name, or any other information we control.

As a result of this complication, our first programming task is to collocate all student submissions from the extracted folder and place them into a single folder. This not only simplifies the rest of our autograder program but also provides a quality of life improvement for the instructional team – when questions arise about a specific submission, the instructional team now knows exactly where to find the original program. Thus, we avoid all manual searches of the cryptically-named folders.

Program 2 shows how we carry out this process via a batch file. Executing the batch file stores a flattened version of the folder structure in a location of our choosing – also defined in Program 1 using the `Out` macro variable. Note that in the following program, as throughout this paper, the red arrows indicate where wrapping has occurred to fit the SAS code into the text box; these line breaks are not present in the original batch file. While creating batch files can be done without SAS, it is an integral part of our file management process because each team member that runs the autograder must have their own copy of the batch file. As such, we leverage SAS to directly create the file within the autograder program. For more reading on batch files, refer to [1] and [4] in the Recommended Reading section.

Program 2: Creating a Batch File to Relocate Submissions

```
data _null_;
  attrib txt length = $ 200;

  file "&out.\Flatten.bat"; ❶

  put "set dSource=&dir"; ❷
  put "set dTarget=&out."; ❷
  put "set fType=&Assignment.*.sas"; ❸
  txt = cat('for /f "delims=" %%f in (', "'dir /a-d /b /s ", '"%dSource
     ↪ %\%fType%"',"') do ("); ❹ ❺
  put txt;
  txt = 'copy "%%f" "%dTarget%\" 2>nul'; ❺ ❻
  put txt;
  put ")"; ❼
  put 'exit'; ❼
run;

x "cd &out."; ❽
x "Flatten.bat"; ❾
```

❶ This null `DATA` step is used to create a batch file, `Flatten.bat`, that is placed in the `Out` directory.

❷ We use `PUT` statements to write to the file named in the earlier `FILE` statement. The `SET` command defines a regular (environment) variable in the batch file. Here, we define two regular variables using SAS macro variables: location of student submissions and location for storing results.

❸ Store a pattern we require the file name to match in order to copy to the new folder structure. Since `*` is a DOS wildcard character, `fType` is matched by any file name that begins with our assignment prefix (via `&Assignment.`), then has any other characters, and then has a .sas extension.

❹ Creates a SAS variable, `txt`, via an assignment statement using the `CAT` function to concatenate strings. This builds a for-loop in the batch file to search for programs we want to copy. The default parser will treat spaces and tabs as delimiters, but since we may encounter files with space in the file name, we use `"delims="` to parse the list with no delimiters, causing each file name to be used as a single token. We search through filenames only (`/a-d`) in the target folder or its subfolders (`/s`) and display only the filename with no metadata such as file size (`/b`).

❺ Note that regular variables in the batch file are referenced using the `%variable-name%` syntax. For-loop variables are referenced using the `%%variable-name` notation and we use quotes to handle files that contain spaces.

❻ This assignment statement, simpler than the previous one, copies the files matching our search parameters to our new location. The option `2>nul` turns off the error messaging – if a file is not copied, we do not generate an error message.

❼ The batch file is completed by closing the parentheses of the for-loop and then exiting the batch file.

❽ Use the `X` statement to change working directory to the location of `Flatten.bat`. This is necessary as a standalone statement before executing the file at our university as most of our network drives utilize UNC paths, which caused issues when issuing these commands simultaneously.

❾ Use a separate `X` statement to execute `Flatten.bat`.

Program 2 produces the batch file shown in Output 2 below.

Output 2: Flatten.bat Batch File to Relocate Submissions

```
set dSource=S:\Documents\Example\Results
set dTarget=S:\SEASUG 2026\Autograder\Demo
set fType=HW00*.sas
for /f "delims=" %%f in ('dir /a-d /b /s "%dSource%\%fType%"') do (
copy "%%f" "%dTarget%\" 2>nul
)
exit
```

Now that all submissions are collocated, we use the code shown in Program 3 to scan the folder, determine the names of the files, and save them for later use. In Program 4 we use `PROC SQL` to create a macro variable based on these file names. The decision to create a data set, rather than using `DATA _NULL_` is intentional to help us with debugging when students do not follow the provided naming guidelines. However, an alternative approach to the `DATA` plus `SQL` combination is possible; for example, using functions like `CATX` and the `CALL SYMPUTX` routine.

Program 3: Creating a List of All Student Programs

```
data StudentPrograms(keep = fname);
  length fref $25 fname $200;
  nameDID = filename(fref,"&out."); ❶
  openDID = dopen(fref); ❷
  if openDID gt 0 then do i = 1 to dnum(openDID); ❸;
    fname = dread(openDID,i); ❹
    output; ❹
  end;
  closeDID = dclose(openDID); ❺
  remDID = filename(fref); ❻
run;
```

❶ Assign the path stored in the macro variable `Out` to a *fileref* stored in `Fref`. `FILENAME` returns 0 if successful and an error code otherwise.

❷ Open the *fileref* using the `DOPEN` function. `DOPEN` returns 0 if unsuccessful and otherwise assigns an identifier to the directory.

❸ If the directory was successfully accessed (`openDID>0`), then we iterate across all files in the directory by using `DNUM` to determine how many files are present.

❹ The `DREAD` function reads the name of the current directory member. We store this in the `Fname` variable and then output the result to the current data set.

❺ Close the connection to the *libref* identified by `openDID`.

❻ When only using one argument, the `FILENAME` function clears the associated *libref*.

At the conclusion of the `DATA` step in Program 3, we have a data set containing a list of submitted file names, i.e., the files that need to be standardized, executed, and then have their logs scanned. Program 4, shown below, creates a single macro variable containing all submission names. We can then scan this macro variable, looping across each item, to identify submissions that need to be run through the autograder. This process is the focus of the sections below titled Pre-Execution Code Editing: Handling Paths and Comments and Executing SAS Programs and Indexing all Logs.

Program 4: Adding the Program List to a Macro Variable

```
proc sql noprint;
  select fname
  into :prgmz separated by "|"
  from studentprograms
;
quit;
```

Pre-Execution Code Editing: Handling Paths and Comments

Before executing the submissions, we make several modifications related to file paths and commented sections of code. The first change to the submissions is a removal of all kernel comments – i.e., comments using the `/*comment*/` style. This helps simplify the log scanning done in later portions of the program by preventing these comments from ever appearing in the log. However, students do not always follow the suggested commenting guidelines so there are multiple ways these may appear in the submission: as part of a single line of code where the comment(s) may be before/after/between the executable code or as part of a multi-line comment. In cases where multi-line comments exist, students use a variety of styles which may or may not include code appearing on the same line where the comment begins/ends. As a result of these uncertainties, we employ several methods to remove these from the program before execution. As Program 5 shows, we use a mix of regular expressions, SAS functions, and conditional logic to handle all possible cases.

Program 5: Removing Comments in Student Programs

```
%macro FixDriveMapping;
  options dlcreatedir; ❶
  libname fixd "&out.\FixedMapping"; ❶
  options nodlcreatedir; ❶

  %let iter = 1; ❷
  %let fname = %scan(&prgmz,&iter,'|'); ❷

  %do %until(%bquote(&fname) = ); ❸
    data _null_;
      infile "&out.\&fname." truncover length = reclen; ❹
      input _line $varying32767. reclen; ❹

      retain _str kcFLag;

      if missing(_line) then delete; ❺
```

Program 5 cont.

```
kc=prxparse("s/\/\*.*?\*\///"); ❻
_line=compbl(prxchange(kc,-1,_line)); ❻

kcstartpos = index(_line,'/*'); ❼
kcendpos = index(_line,'*/'); ❼

if kcstartpos eq 1 and kcendpos eq 0 then kcflag=1; ❽
  else if kcstartpos gt 1 and kcendpos eq 0 then
    do; ❽
      _str = substr(_line,1,kcstartpos-1);
      kcFlag = 1;
    end;
  else if kcstartpos eq 0 and kcendpos gt 0 then
    do; ❽
      kcFlag = 0;
      _line = cat(trim(_str),substr(_line,kcendpos+2));
      call missing(_str);
    end;

if kcFlag eq 1 then delete;  ❾
```

❶ Create a folder, FixedMapping, to hold the edited submissions.

❷ Prepare to scan through the pipe-delimited list of program names by determining the first file name.

❸ Loop until no more student programs are found. The `%BQUOTE` function is essential as some students' names contain characters like hyphens that the macro compiler may interpret as code.

❹ The `INFILE` statement is used to read the code contained in a submission and determine the length of the line of code. The `$VARYING.` informat allows us to read the varying-width lines of code.

❺ Delete any blank lines to improve readability of the final code as some students include excessive blank space throughout their programs.

❻ We use regular expressions to identify any lines that contain a kernel comment such as `/*this is a comment*/` and replace those comments with a null string. For more information on Perl regular expressions and their implementation in SAS, please see [5].

❼ Now that all single-line kernel comments are removed, these two assignment statements determine the position of starting and ending tags for kernel comments that span multiple lines.

❽ If the line includes the beginning of a kernel comment, then use `SUBSTR` to extract all code prior to the comment and flag the line for deletion. If the line includes the end of a kernel comment, use `SUBSTR` to extract all code after the closing comment tag. Then use `CAT` to concatenate the code snippets from before and after the comment, creating a single complete statement with the comment removed. At this point, set the kernel comment flag to 0 to keep the line in the final file.

❾ Delete lines of code if they were flagged for deletion during the removal of kernel comments.

With all comments removed, our second order of business at this point is to address the issue of file paths. Despite writing and executing their code on a centralized computing environment, students have the flexibility to organize their files however they like. As a result, for us to execute their code, all file paths must be standardized. Even though their files may be arranged however they like, we do require that all file creation (e.g., documents, images, data sets) is done relative to their working directory. Thus we only have to edit their code to replace their working directory declaration with the path of our choosing. Our path is partly defined by the macro variable `Out` in Program 1; we add a new folder to avoid overwriting a submission with our edited version. Note that Program 6 is a continuation of the `DATA` step, and thus the macro, introduced in Program 5.

Program 6: Redirecting Pathing in Student Programs

```
    if index(lowcase(_line),'ods trace') ge 1 then delete; ❶
      else if lengthn(compress(_line)) ge 7 then
        do; ❷
          if substr(compress(lowcase(_line)),1,6) in ('x"cds:' 'x''cds:'
             ↪ 'x"cdb:' 'x''cdb:' 'x"cdc:' 'x''cdc:') then
              do; ❸
                _line = cat('x "cd ', "&out.\FixedMapping", '";'); ❹
                len = reclen+lengthn("&out.\FixedMapping"); ❺
                file "&out.\FixedMapping\&fname."; ❻
                put _line $varying32767. len; ❻
              end;
            else
              do;
                file "&out.\FixedMapping\&fname."; ❻
                put _line $varying32767. reclen; ❻
              end;
        end;
      else
        do; ❻
          file "&out.\FixedMapping\&fname.";
          put _line $varying32767. reclen;
        end;
    run;

    %let iter = %eval(&iter+1); ❼
    %let fname = %scan(&prgmz,&iter,'|'); ❼
  %end;
%mend FixDriveMapping;

%FixDriveMapping
```

❶ Delete lines that contain `ODS TRACE` to avoid unnecessary information in the log.

❷ To avoid searching every line of code, we only check lines that are long enough to contain a valid `X` statement with an absolute path – the only way our students are allowed to change their working directory. All possible drives available to students (B, C, and S) are checked.

❸ Any `X` statement pointing to one of these drives is modified to use the path we designate in Program 1 via the `Out` macro variable.

❹ Because the substitution of our path into the line may make it longer, we recompute the line length.

❺ The `FILE` statement designates the file we want to create – note the use of the `Fname` macro variable to ensure we preserve the student's name. The `PUT` statement writes the current line of code into this file and this is done whether or not the line is modified.

❻ Update the iterator variable and identify the next name from the pipe-delimited list, similar to the approach in ❷ of Program 5.

After successfully building the `FixDriveMapping` macro, we immediately execute the macro. This creates the alternate versions of the submissions with each submission now containing no blank lines, no kernel comments, no `ODS TRACE` statements, and no directories that the instructional team cannot access. At this point, we are ready to execute all programs so that we can save, and ultimately scrape, all of the logs.

Executing SAS Programs and Indexing all Logs

Once the programs have been modified so that they can be run in the grader's environment, the next step is to execute the files and save all relevant output. Program 7 follows a similar pattern to the earlier batch file creation; however, this time the batch file executes each SAS file and saves the expected output, namely the corresponding log and listing files. To execute SAS from the batch file, we use the path to the SAS executable stored in the `SasExeDir` macro variable defined in Program 1. Each program is launched in a unique environment, so that any errors or settings applied in one do not carry over to other submissions. This also ensures that each student's work is tracked independently which allows for subsequent automatic review without information from one submission affecting another. Following the batch file creation, the file is run using the same process as described in Program 2.

Program 7: Creating Batch File for Cleaned Programs

```
data _null_;
  attrib txt length = $ 200;
  set StudentPrograms end=eof; ❶

  file "&out.\FixedMapping\RunStudentPrograms.bat";

  if _n_=1 then do; ❷
      put "set SASLOC=&SasExeDir."; ❸
      txt='set PRJLOC='||strip("&out.")||"\FixedMapping"; ❹
      put txt; ❹
      put 'CD %PRJLOC%'; ❺
      put 'DEL /Q *.LOG'; ❺
      put 'DEL /Q *.LST'; ❺
    end;

  txt='"%SASLOC%\sas.exe" -rsasuser -noicon -nosplash -nosyntaxcheck
     ↪ -nostatuswin -nologo -SYSIN "%PRJLOC%\'||strip(fname)||'"'; ❻
  put txt; ❻

  if eof then put 'EXIT'; ❼
run;
```

❶ Read in the data set containing the names of the submitted files. Use `END=` to keep track of the last record to facilitate the conditional processing in ❾.

❷ This block of code is only executed during the first iteration to write code to the batch file that should be executed *before* the submitted file names are scanned.

❸ Create a regular batch variable containing the SAS installation location. This was done in Program 1 by applying the `%SYSGET` function, prompting Windows for the SASROOT directory's location.

❹ Create and print to the batch file a regular variable that contains the current project folder's location.

❺ To ensure a clean environment each time the log scan program is executed, we delete all pre-existing .log and .lst files from the project location.

❻ For each file name, we create a text string that launches SAS.exe with several system options: prevent write-access to the SasUser library, prevent the SAS session from being minimized, suppress the SAS logo splash screen, suppress the status window during batch mode, prevent SAS copyright/version info from appearing in the log, and submit the named file in batch mode. After these options, the current project location is concatenated with the current file name from the data set.

❼ Finally, once we finish writing a batch command for the last submission, we use conditional logic to detect the end of file and print an `EXIT` statement to the batch file. This allows the batch session to end after all student submissions are executed.

Scanning Logs

Executing the batch file written in Program 7 runs each submission and stores the logs. With the logs generated, we prepare to scan them to identify features of interest. Because this review process must be applied to every submission, we use the same approach as used in Program 3 and Program 4 to create a list of generated log files. As in the earlier programs, this list allows us to iterate through the logs and apply the same review criteria to each submission. The `DATA` and `SQL` steps to achieve this are shown in the full code but omitted here due to their similarity to Programs 3 and 4.

Before actually reviewing log files, we remove results from any previous execution of the log scanner to prevent any confusion across assignments. Program 8 shows the beginning of the log scanning macro, focusing on some housekeeping before we begin scanning the logs for features of interest. This macro continues through Program 15.

Program 8: Initial Checks Prior to Scanning

```
%macro StudentLogScan(debug=no); ❶
  %let iter = 1; ❷
  %let fname = %scan(&logz.,&iter.,'|'); ❷

  %if %sysfunc(exist(&Assignment.StudentLogScans)) %then %do; ❸
    proc datasets; ❸
        delete &Assignment.StudentLogScans / memtype = data; ❸
      run;
    quit;
  %end;

  %do %until(%bquote(&fname) = ); ❹
    data LogScanTemp;
      infile "&out.\FixedMapping\&fname." truncover length = reclen; ❺
      input _line $varying32767. reclen; ❺

      _line = lowcase(_line); ❻
      if index(_line,'the sas system') eq 0 and lengthn(_line) gt 0; ❻
    run;
```

❶ Define the `StudentLogScan` macro with a single, optional parameter. The optional debug argument is used later in the code to keep rather than drop intermediate variables when debugging is desired.

❷ Identify the first log in the pipe-delimited list stored in the macro variable `logz`. As described earlier, this variable is created similar to `prgmz` in Program 4.

❸ Check whether a results data set already exists for the current assignment, and if so, delete it. This ensures that results from an earlier execution are not combined with the current review.

❹ Loop through the list of log files until no additional filename is found. As in Program 5, `%BQUOTE` prevents characters within filenames from being interpreted by the macro processor.

❺ Read each line of the current log into the temporary variable `_line`. The `$VARYING.` informat accommodates log lines of different lengths.

❻ Convert each line to lowercase so subsequent text searches don't need to be case sensitive, then remove blank lines and standard SAS page headers that do not contain information needed for the code review.

For each file, we start by identifying all the aspects which we are interested in tracking and initialize how they should be stored. Some items we are interested in are a simple count of keywords while others are more complex. For example, we aim to differentiate between types of syntax errors and penalize only a subset of 'bad' notes, not all notes found in the log.

Program 9 shows how we approach this. Arrays defining our criteria allow for easier changes in the future, should the assessment criteria change, without requiring a rewrite of the scanning logic. We also define several variables needed throughout the review process. In particular, the `END=` option creates the variable `last`, which identifies the final line of the current log. Later, this will allow us to retain only the complete summary record for each student submission.

Program 9: Initializing Necessary Components

```
data LogScan %if (&debug = no)
             %then (drop = _: OpOnMissFlag ImpConvFlag QuitCount
                            RunCount StepStart);; ❶

  length Student $50 _col1to5 $5 GlobalStrings $32767;

  Student = scan("&fname.",1,'.'); ❷
  set LogScanTemp end = last; ❷

  retain UninitCount ImpConvCount LostCardCount OpOnMissCount 0
         ImpConvFlag OpOnMissFlag 0
         UnrecovSyntaxErrCount RecovSyntaxErrCount 0
         TrueWarnCount 0
         StepStart GlobalStatementCount 0
         GlobalStrings
  ; ❸

  array SingleNoteCountz[2] UninitCount LostCardCount; ❹
  array SingleBadNotez[2] $ 50 _temporary_
      ('is uninitialized' 'lost card'); ❹
  array MultiNoteCountz[2] ImpConvCount OpOnMissCount; ❹
  array MultiBadNotez[2] $ 50 _temporary_
      ('have been converted' 'missing values were generated'); ❹
  array MultiFlagz[2] ImpConvFlag OpOnMissFlag; ❹
  array BadErrz[3] $ 50 _temporary_
      ('syntax error' 'unrecognized sas option' 'was misspelled'); ❺
```

❶ Create the temporary results data set. Unless the macro is executed in debug mode, intermediate variables used only during the log scanning are dropped from the resulting data set.

❷ Identify the student from the current log filename and read the cleaned log. The `END=` option creates `last`, which identifies the final line and will later be used to retain one complete summary observation per submission.

❸ Initialize and retain the counters and flags used throughout the log review. Retaining these variables allows information detected on different lines of the log to be accumulated within the submission.

❹ Define parallel arrays containing the text associated with concerning SAS notes and the variables used to count these occurrences. Separate arrays are used to handle each of the ways that SAS reports notes: either one NOTE message for each occurrence or one NOTE message that summarizes multiple occurrences. Flags are included for the latter case to connect the type of note with the subsequent line containing the locations of the individual occurrences.

❺ Store the text patterns used to identify the syntax-related errors and warnings of interest. Storing these criteria in an array allows the set of tracked messages to be modified without changing the subsequent scanning logic.

We now begin the actual assessment of the student's log by checking for concerning notes, as defined in Program 9. As mentioned above, the various ways SAS reports notes requires us to develop two strategies for counting these problematic notes.

Since we are interested in the number of occurrences, not NOTE messages, we must extract the number of impacted locations. We then combine these counts and save the total number of concerning notes to the overall data set. Program 10 shows how we leverage regular expressions to process the logs to handle the counting of problematic notes.

Program 10: Checking Log for Concerning Notes

```
_notePattern = prxparse("/note:/"); ❶
_noteMatch = prxmatch(_notePattern,_line); ❶

_multiPattern = prxparse("/\d+:\d+/"); ❷
_multiMatch = prxmatch(_multiPattern,_line); ❷

do _i = 1 to dim(SingleBadNotez); ❸
  if (_noteMatch gt 0) and (index(_line,strip(SingleBadNotez[_i])) gt 0)
    then SingleNoteCountz[_i] + 1; ❸
end;❸

do _i = 1 to dim(MultiBadNotez); ❹
  if (MultiFlagz[_i] eq 1) and (_multiMatch gt 0) then
    do; ❹
      MultiNoteCountz[_i] + countc(_line,":"); ❹
      MultiFlagz[_i] = 0; ❹
    end; ❹
  if (_noteMatch gt 0) and (index(_line,strip(MultiBadNotez[_i])) gt 0)
    then MultiFlagz[_i] = 1; ❹
end; ❹

BadNoteCount=sum(UninitCount,ImpConvCount,OpOnMissCount,LostCardCount);❺
```

❶ Create and apply a regular expression that identifies lines containing SAS NOTE messages.

❷ Create a second regular expression that identifies the line-location pattern SAS uses when reporting multiple occurrences within a single NOTE.

❸ For notes that SAS reports separately for each occurrence, search for each predefined text pattern and increment its corresponding counter whenever a match is found.

❹ For notes that summarize multiple occurrences, use a flag to connect the initial NOTE message with the subsequent line containing the affected locations. Count those locations and then reset the flag before continuing through the log.

❺ Sum the individual note counts to create an overall count of concerning NOTE messages identified in the current submission.

Catching errors and warnings follows a similar process. We don't differentiate between warning types, so all warning messages contribute to a single count. For syntax-related messages, however, we differentiate between recoverable and unrecoverable errors. We therefore maintain separate counts for these two cases. This is complicated by the fact that in the case of a recoverable syntax error – e.g., SAS detects a misspelled keyword and fixes it for you – SAS only issues a WARNING message and not an ERROR message. As such, Program 11 must not only scan for the ERROR and WARNING messages, it must determine if a WARNING message is actually the result of a syntax error.

Program 11: Checking Log for Errors and Warnings

```
_errPattern = prxparse("/error\s\d+-\d+:/"); ❶
_warnPatternLong = prxparse("/warning\s\d+-\d+:/"); ❶
_warnPatternShort = prxparse("/warning:/"); ❶

_errMatch = prxmatch(_errPattern,_line); ❶
_warnMatchLong = prxmatch(_warnPatternLong,_line); ❶
_warnMatchShort = prxmatch(_warnPatternShort,_line); ❶

if (_errMatch gt 0) and ((index(_line,strip(BadErrz[1])) gt 0) or
                         (index(_line,strip(BadErrz[2])) gt 0)) ❷
      then UnrecovSyntaxErrCount + 1 ❸;
  else if (_warnMatchLong gt 0) and (index(_line,strip(BadErrz[3])) gt 0)
    then RecovSyntaxErrCount + 1; ❹
  else if _warnMatchShort gt 0
    then TrueWarnCount + 1; ❺
```

❶ Define and apply regular expressions corresponding to the standard forms of SAS error and warning messages and determine whether the current line matches each form.

❷ Use these regular expressions and the error text from Program 9 to distinguish unrecoverable and recoverable syntax errors. ERROR messages with any text string we flag are unrecoverable errors.

❸ Once we determine the type of message we increment the corresponding counter.

❹ If the message is a WARNING message related to a misspelled keyword, it is a recoverable syntax error and we increment the associated counter.

❺ If the line contains a WARNING message that was not already classified as a recoverable syntax error, increment the general warning counter.

We also check the students' code for good programming practices. Two such practices are efficient coding and placing correct boundary statements throughout their code. To teach students efficiency, each assignment is given with a specific budget of PROC and `DATA` steps within which students should be able to solve the problem at hand. Program 12 demonstrates how we collect this information. We know that students must start each `PROC` or `DATA` step with the respective keyword and that each must end with a `run;` statement. This class does not utilize procedures which also require a `quit;` to fully terminate, so the only `quit;` statement which should appear is at the end of the students' code.

In order to check this, we examine each line to validate that it is long enough to be worth examining and then subset the line to select the first five characters, as this is sufficient to uniquely identify all of the keywords we are interested in. We then keep a running tally of each keyword of note. The tallies for `procCount` and `dataCount` are saved so that the grader can check them against the approved budget for this assignment. There is also a boundary difference calculation which summarizes how many incorrect boundary situations arise in the code. The errors could be too many or too few of either type of boundary statements for the contents of the code.

Program 12: Checking the Log for Appropriate Boundary Statements

```
if (lengthn(compress(_line,' ','d')) ge 4) ❶and
   (substr(left(_line),1,8) ne 'dataname')❶
      then _col1to5 = substr(compbl(left(compress(_line,,'d'))),1,5); ❷

select(_col1to5); ❸
  when('proc') procCount+1; ❹
  when('data') dataCount+1; ❹
  when('run;','run ;') runCount+1; ❺
  when('quit;', 'quit') quitCount+1; ❺
  otherwise;
end;

BoundDiff = abs(procCount+dataCount-runCount) + abs(quitCount-1); ❻
```

❶ Confirm that the line is eligible to be the beginning or end of a step by ensuring that it has sufficient length. To avoid confusion with `ODS TRACE` output on identifying a `DATA` step, we also purposefully check against the line beginning with `dataname`.

❷ For approved lines, extract the beginning of the submitted SAS statement. Five characters are sufficient to distinguish between the `PROC`, `DATA`, `RUN`, and `QUIT` statements needed for this process.

❸ Use a `select` block to increment the correct counter when the beginning or end of a `DATA` or `PROC` step is identified.

❹ For the beginning of a step, we ensure the statement starts with `DATA` or `PROC` followed by a space. Note that by default, string comparisons in SAS ignore trailing spaces so the `WHEN` statements do not need to include the trailing spaces for valid comparisons.

❺ While spaces are not expected between `RUN` and `QUIT` statements and their semicolons, we do have cases wehre students have included them. As a result, we must consider both cases. The `COMPBL` function in ❷ ensures we can have at most one space between the keyword and the semicolon, simplifying our logic.

❻ Compare the number of `DATA` and `PROC` steps with the number of `RUN` statements and verify the number of `QUIT;` statements. As we don't use `PROC` steps in our class that expect a `QUIT;` statement mid-program, we subtract off 1 to account for the single `QUIT;` at the end of the program. `BoundDiff` summarizes discrepancies in the boundaries, with a value of zero indicating the expected balance.

Another of the good programming practices we assess is that global statements should appear outside of any step. Therefore, we need to track not only what statements are written but where they appear in the code. We begin by checking whether the code has entered into a `PROC` or `DATA` step. If so, we count the number of lines which start with any of the keywords associated with the global statements covered in the course. Once the current section of code ends, the counting stops and restarts upon entering the next section.

Program 13: Checking the Log for Global Statement Location

```
cond1 = compress(lowcase(scan(_line,2,' ')),' ;'); ❶
cond2 = compress(cond1,,'d'); ❶

if _col1to5 in ('data' 'proc') then StepStart = 1; ❷
  else if cond1 in ('run' 'quit') then StepStart = 0; ❷

if StepStart eq 1 and
    (cond2 in ('title' 'footnote') or
     cond1 in ('ods' 'options' '%let' 'x' 'filename' 'libname' 'dm'))
  then do; ❸
      GlobalStatementCount + 1; ❸
      _str=substr(compbl(_line),1,min(lengthn(compbl(_line)),25)); ❹
      globalStrings = catx("200A20"x,globalStrings,_Str); ❹
    end;
```

❶ Extract and standardize the beginning of each submitted line. Two versions are retained because statements such as `TITLE` and `FOOTNOTE` may have numeric suffixes, while the remaining global statements can be matched directly.

❷ Track whether the current line occurs within a `DATA` or `PROC` step. Encountering either the DATA or PROC keyword turns the flag on, while RUN and QUIT turn them off. Recall `_col1to5` was created in Program 12.

❸ When inside a step, check whether the current line begins with one of the global statements covered in the course and increment the counter when one is found.

❹ Save a shortened version of each misplaced global statement in addition to counting it. This provides the grader with enough information to identify the offending statement without requiring a manual search through the log or code.

This `DATA` step was concluded by subsetting this data using `last eq 1`. Last was defined in Program 9 which allows us to have one summary line of all information for a given student's assignment. This completed summary is then appended to the results data set for the assignment. We then advance to the next student's log and repeat the scanning process. The loop continues until every submission has been reviewed.

Program 14: Creating the Results Data Set

```
proc append base = &Assignment.StudentLogScans
            data = LogScan; ❶
run;

%let iter = %eval(&iter+1); ❷
%let fname = %scan(&logz,&iter,'|'); ❷
%end;
```

❶ Append the completed summary for the current student to the assignment-level results data set.

❷ Advance the iterator and retrieve the next log filename. The macro loop then repeats the review process until no additional filename is found.

Building and Reporting the Code Review Summary

Once we collate information from all submissions into a single data set, the final step is creating results the instructional team can use easily. We use `PROC REPORT` and the `ODS EXCEL` destination to create a grader-facing summary. For readability, related review criteria (i.e. errors or notes) are grouped by common header color. Additionally, we use `PROC DATASETS` to assign descriptive labels before producing the report. Program 15 shows a truncated version of the code used to generate the final Excel workbook. Excel was selected both because it provides an accessible format for the graders and because the resulting file can be retained as a record of the automated review in the event of a regrade request.

Program 15: Printing Results to Excel

```
  ods exclude all; ❶
  ods excel file = "&out\FixedMapping\&Assignment.Results.xlsx"; ❶

  proc report data = &Assignment.StudentLogScans split ='0A'x; ❷
    column Student
           UnrecovSyntaxErrCount RecovSyntaxErrCount TrueWarnCount
           BadNoteCount OpOnMissCount ImpConvCount UninitCount
              ↪ LostCardCount
           BoundDiff dataCount procCount GlobalStatementCount
              ↪ GlobalStrings; ❸
    define Student / display style(column) = [width=2in]; ❹
    define UnrecovSyntaxErrCount / display style(header) = [
       ↪ backgroundcolor = cx700000 color = cxFFFFFF]; ❺
        *Other define statements removed for brevity;
    define GlobalStrings / display style(header) = [backgroundcolor =
       ↪ cxFDC086 color = cxFFFFFF width=4in] ❻;
  run;

  ods excel close; ❶;

%mend StudentLogScan; ❼

options nosymbolgen nomlogic; ❽
%StudentLogScan(debug=yes) ❽
```

❶ Prevent these results from appearing in any currently open destination. Use an Excel destination for the current assignment's code review results.

❷ Use PROC REPORT to create the grader-facing summary. The hexadecimal line-feed character is used as the split character so that longer column headings can be displayed across multiple lines.

❸ Specify the review measures included in the report and their display order, grouping related information together for easier review.

❹ Allocate additional column width to keep identifying information and review details readable.

❺ Apply common header formatting to related review measures using a format, so that different categories can be visually distinguished. The complete program applies these definitions to all reported variables; selected statements are shown here for brevity.

❻ Provide additional width for `GlobalStrings`, which contains excerpts of misplaced global statements and therefore requires more display space than the numeric summary columns.

❼ Conclude the `StudentLogScan` macro that we began in Program 8.

❽ Note that calling a macro should not require a semicolon, so none is included here.

Conclusion

Code reviews can be a daunting task as the number of files to be reviewed grows. Automating the reviews allows for a reduction in the amount of time needed for low-level tasks like executing code, scraping the log for standard terminology, and collating results. In this paper we have laid out a possible solution that employs a wide variety of coding tools both within SAS and beyond it.

As laid out here, our suggested workflow is as follows:

1. Ensure files to be simultaneously assessed are stored in a centralized location. [Programs 1 and 2]
2. Create a list of all files to be assessed. [Programs 3 and 4]
3. Standardize all file contents to meet predefined specifications such as file paths, file formatting, etc. [Programs 5 and 6]
4. Execute all programs and save the logs. [Program 7]
5. Scan the logs for all features of interest. [Programs 8 through 13]
6. Store and report the results. [Programs 14 and 15]

Due to the nature of the problem – text analytics applied to program logs – the overall program can be somewhat overwhelming at first. By breaking it into chunks we hope to help readers both in terms of motivating the need for each chunk and identifying whether a chunk is applicable to the code review process in their industry. Furthermore, this paper demonstrates several common use cases for arrays, regular expressions, and other programming techniques.

Recommended Reading

[1] *Batch File*. URL: https://en.wikipedia.org/wiki/Batch_file (visited on 10/02/2026).
[2] James Blum and Jonathan Duggins. *Fundamentals of Programming in SAS: A Case Studies Approach*. SAS Press, 2019. ISBN: 978-1635266726.
[3] SAS Institute. *DLCREATEDIR System Option*. URL: https://documentation.sas.com/doc/de/vdmmlcdc/8.1/lesysoptsref/n1pihdnfpj4b32n1t62lx0zdsmdn.htm (visited on 10/02/2026).
[4] SS64. *SS64 Command Line Reference*. URL: https://ss64.com/nt/ (visited on 10/02/2026).
[5] Edwin Xie and John M. LaBore. “Unleash Your Coding Potential: SAS PRX Functions for Next-Level String Manipulations”. In: *PharmaSUG*. 2025.

Contact Information

Your comments and questions are valued and encouraged. Contact the corresponding author at:

Jonathan Duggins

North Carolina State University

jwduggin@ncsu.edu

https://jonathanduggins.com/